DeadlineIQ

Cognitive-Load AI Task Planner & Focus Companion

Technical Documentation

Version 1.0 • June 2026

Author: **Moteesh A**

AI-Powered | Firebase | Gemini 2.5 | React 18

DeadlineIQ Engineering | Confidential

Contents

CHAPTER 1

Product Overview

1.1 Introduction

DeadlineIQ is an ultra-premium, AI-driven productivity companion engineered to defeat procrastination and optimise cognitive focus. Unlike conventional productivity tools that rely on passive push alerts — which users habitually ignore — DeadlineIQ introduces a *proactive, spatial, and cognitively aware* approach to task management.

The system analyses behavioural patterns in real time, predicts task-completion risks using a client-side machine learning model, and orchestrates personalised focus windows directly around the user's existing calendar commitments.

DeadlineIQ replaces reactive reminders with **predictive intelligence** — it intervenes before deadlines are missed, not after. Every feature is designed to reduce cognitive load, not add to it.

1.2 Key Capabilities

1. **Local Neural Network Risk Scoring** — A client-side Multi-Layer Perceptron (MLP) computes personalised procrastination probability scores without transmitting sensitive data to external servers.

2. **Natural Language Task Parsing** — Powered by *Gemini 2.5 Flash*, users dictate chaotic schedules in plain English and receive structured task objects with cognitive-load estimates and confidence scores.

3. **Calendar-Aware Focus Scheduling** — Deep integration with the Google Calendar API allocates focus windows around existing commitments and peak-productivity periods.

4. **Procrastination Forensics** — Recalculates behavioural delays to identify and categorise procrastination root causes (e.g., *Fear of Failure*, *Task Ambiguity*) with SVG trend charts.

5. **Agentic Co-pilot (IQ Coach)** A sliding sidebar AI agent executes scheduling actions, reschedules tasks, and commits all updates to Firestore in real time.

6. **Habit Streak Tracking** Glowing circular gauges monitor routine completions with Gemini-powered success probability forecasts.

7. **Chrome Extension Integration** Synchronises browser research directly to the cloud task dashboard without context-switching.

CHAPTER 2

User Interface Walkthrough

DeadlineIQ's UI is built on a **VisionOS-inspired spatial design language** floating telemetry modules, glassmorphic panels, and a premium dark-mode slate palette that minimises distraction while maximising information density.

2.1 Primary Navigation HUD (Dashboard)

The main dashboard is the command centre of DeadlineIQ. It surfaces live task metrics, multi-agent status, and focus-window data through floating telemetry panels.

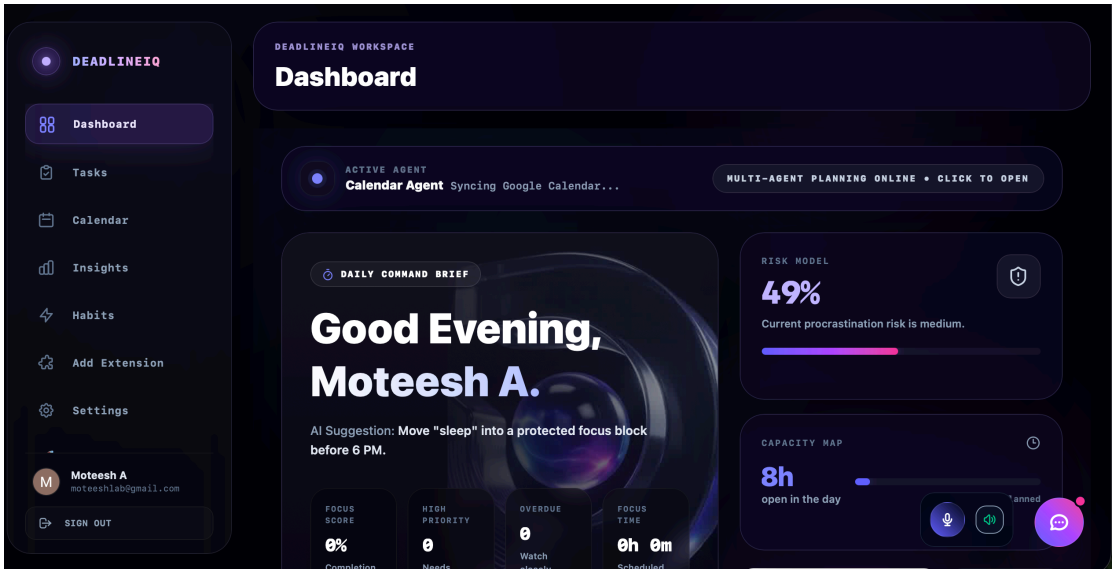


Figure 2.1: Primary Navigation HUD showing floating telemetry modules and live task tracker.

Key UI Elements:

- Floating telemetry cards with live task counts and urgency indicators
- Multi-agent status banner showing active AI operations
- Focus score ring and daily priority hour counter
- One-tap access to all feature modules via the sidebar rail

2.2 Local Neural Network Risk Model

An integrated client-side neural net assesses active task parameters in real time to produce a personalised **Procrastination Probability Score**.

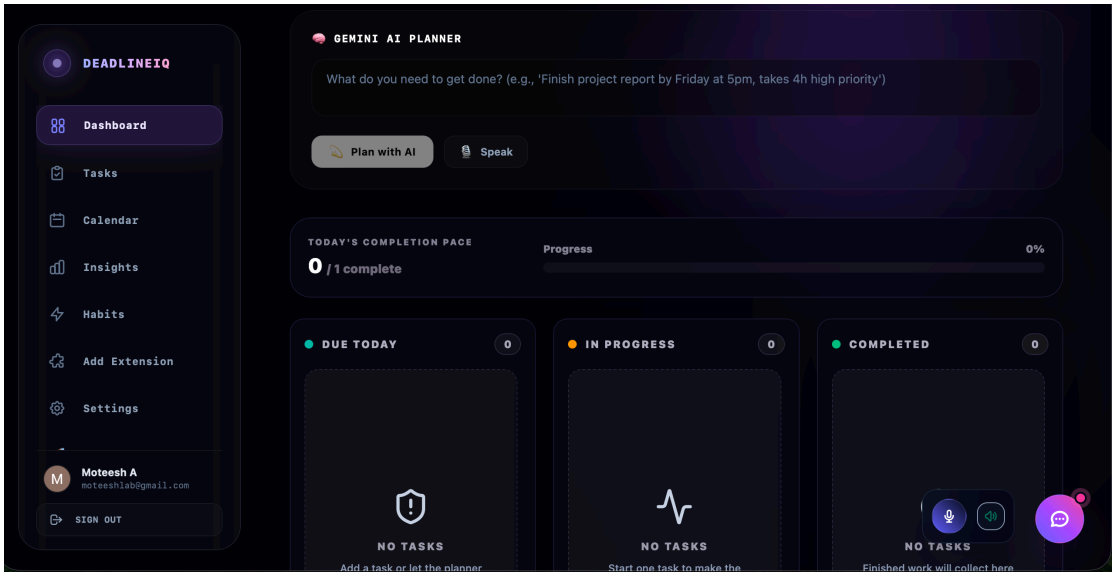


Figure 2.2: Local MLP Neural Network Risk Model Widget displaying real-time task completion probability.

2.3 Capacity Mapping & Focus Nudges

This module displays floating telemetry panels indicating current focus scores, active priority hours, and ambient audio controls that help users enter deep-work states.

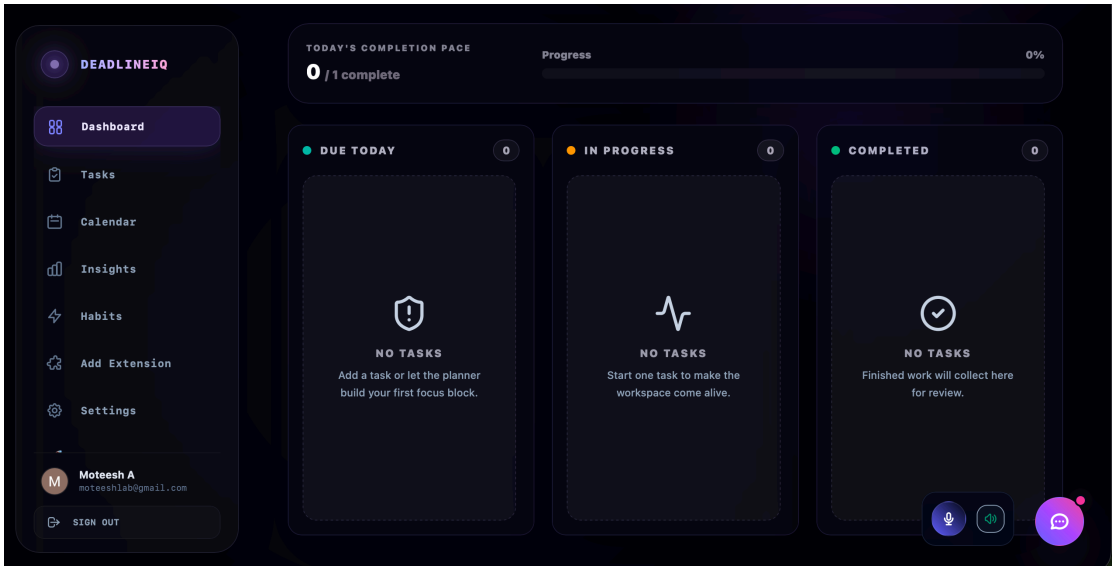


Figure 2.3: Capacity Mapping and Focus Nudge module showing available workload capacity.

2.4 Gemini Natural Language Task Parser

An automated schedule deconstruction module. Gemini 2.5 Flash parses the voice or text input, extracts structured task objects, and estimates cognitive load metrics and confidence scores automatically.

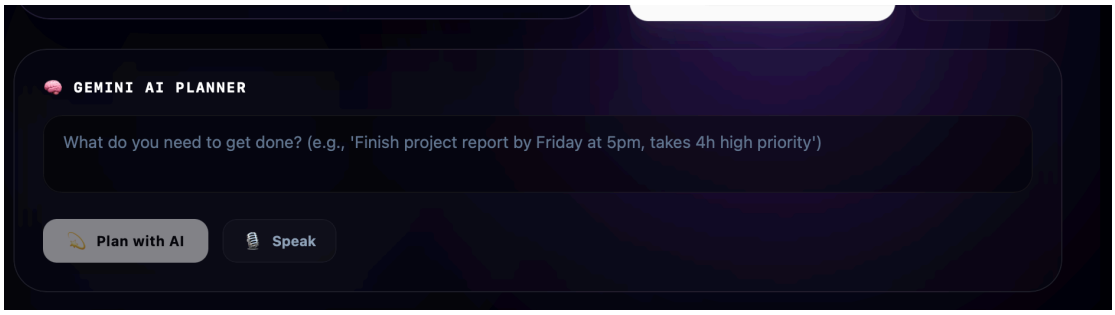


Figure 2.4: Gemini NL Task Parser showing text-to-task deconstruction and semantic structure evaluation.

User input: *"Finish presentation slides by tomorrow at 4 PM, high priority, will take 3 hours."*

Gemini output:

- `title`: Finish presentation slides
- `deadline`: Tomorrow 16:00
- `priority`: High
- `estimated_duration`: 3h
- `cognitive_load`: 7.4/10
- `confidence`: 94%

2.5 Interactive Co-pilot IQ Coach Sidebar

A sliding AI sidebar where users can naturally command, reschedule, and complete tasks. The agent executes all actions and commits updates to Firestore in real time.

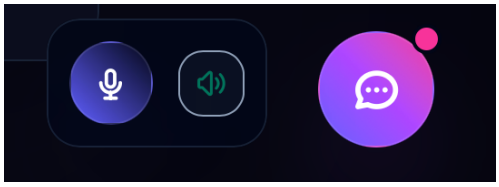
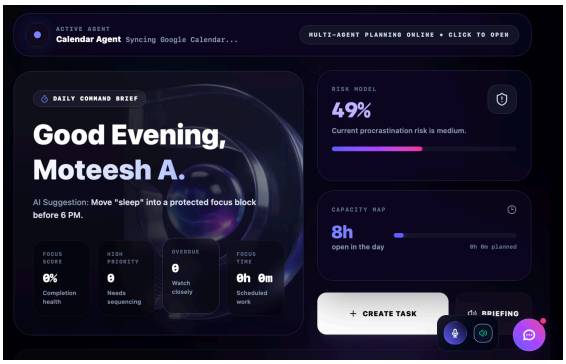


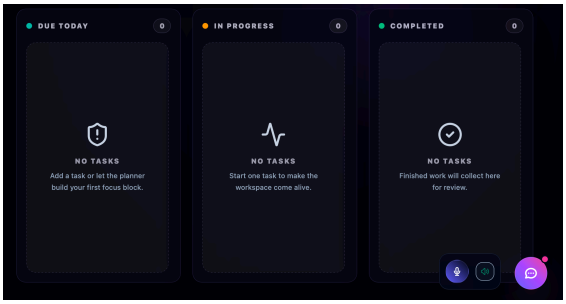
Figure 2.5: Speech integration and co-pilot activation buttons shown at original high-definition scale.

2.6 Task Deconstruction & Subtask Builder

Gemini automatically decomposes complex deadlines into discrete actionable subtasks. Users can edit subtask parameters, adjust time estimates, and track incremental completion.



(a) Task Deconstruction Modality



(b) Subtask Parameter Builder

Figure 2.6: Task Deconstruction and Subtask Builder interfaces

2.7 Kanban Progress Board

A drag-and-drop board for tracking commitments across progress columns. Audio chimes fire on task completion to provide positive reinforcement.

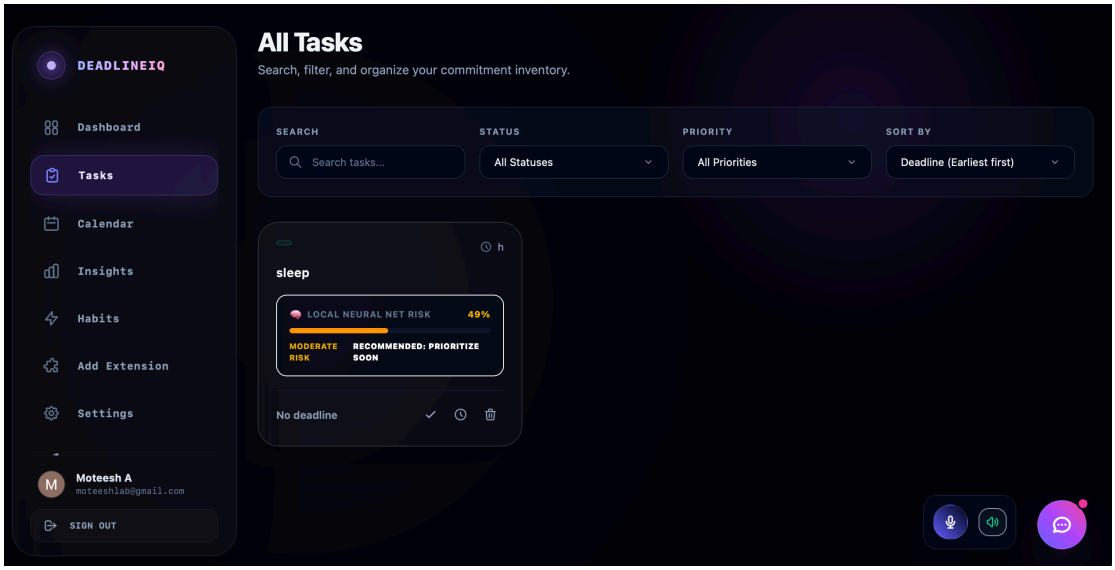


Figure 2.7: Kanban Progress Board with columns: *Due Today*, *In Progress*, *Completed*

2.8 Glassmorphic Calendar Grid

An edge-to-edge borderless weekly calendar that overlays tasks alongside Google Calendar events and highlights Peak Focus Windows for optimal scheduling.

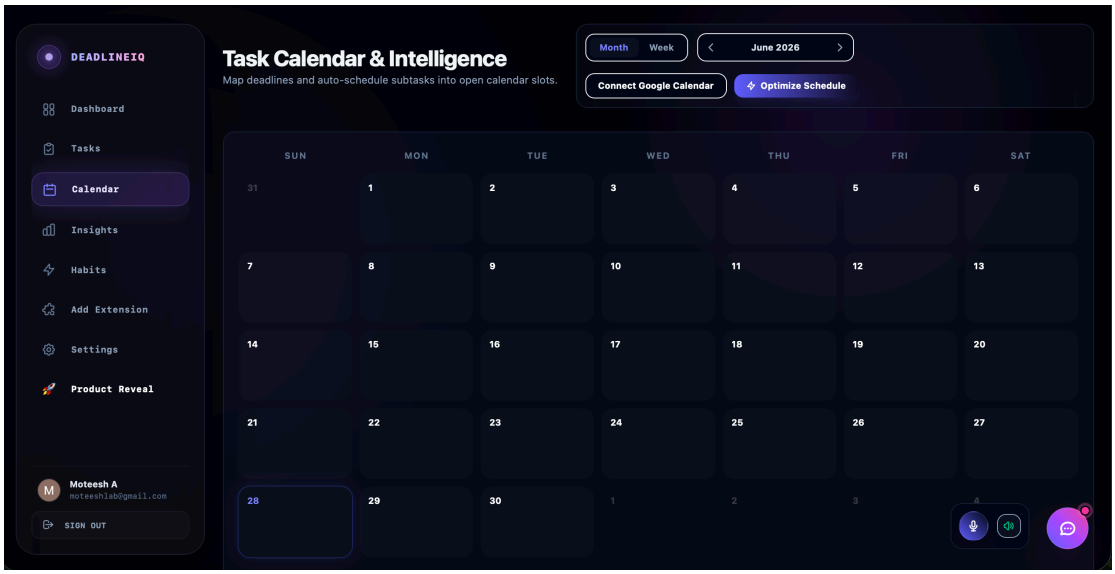


Figure 2.8: Glassmorphic VisionOS Calendar Grid with Google Calendar sync

2.9 Procrastination Forensics & Score Forecasting

This analytics module catalogues behavioural delay patterns to identify the user's *procrastination fingerprint* and charts predictive score trends with SVG visualisations.

Figure 2.9: Procrastination Forensics and Predictive Score Dashboard

2.10 Habit Streak Tracker

Monitors routine completions with glowing circular progress gauges and integrates Gemini-powered success probability forecasts for each tracked habit.

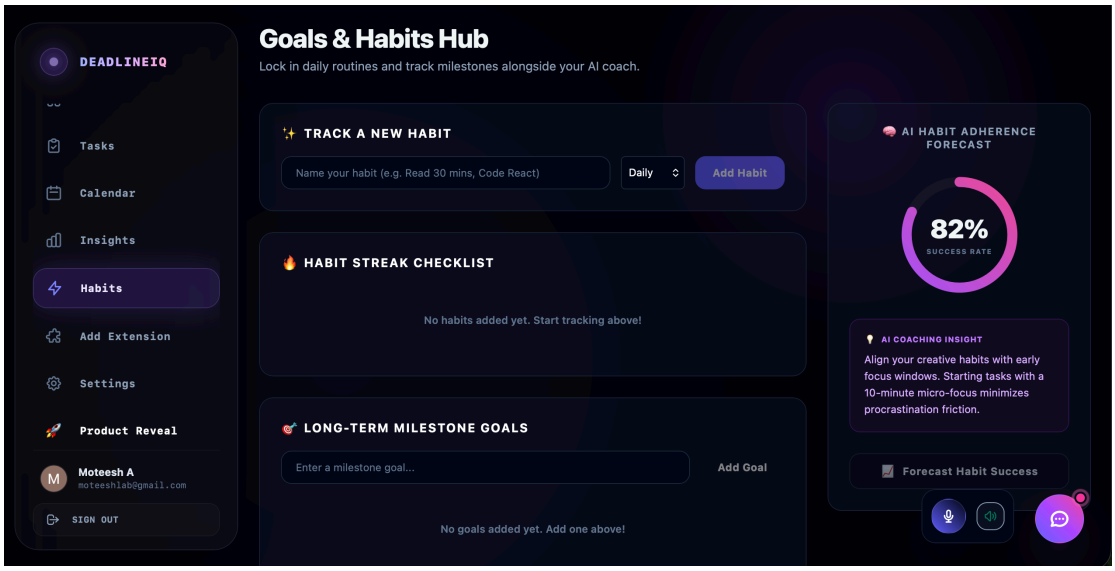


Figure 2.10: Habit Streak Tracker with circular progress gauges

2.11 Chrome Extension Panel

The companion Chrome extension synchronises web research tabs, captures snippets, and pushes tasks directly to the cloud dashboard eliminating context-switching.

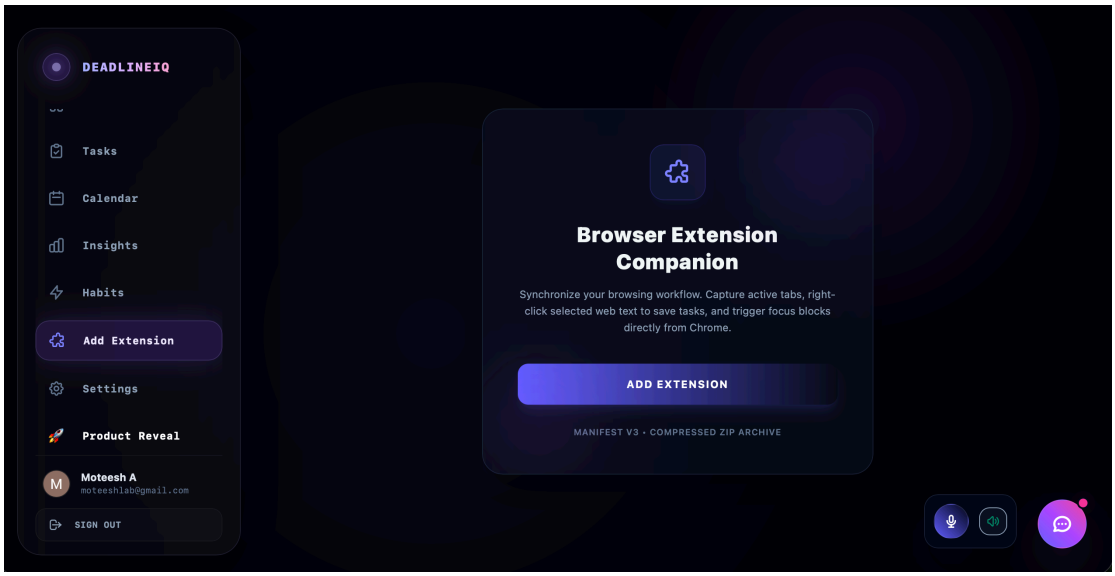


Figure 2.11: Chrome Extension Panel browser-to-dashboard task pipeline

CHAPTER 3

Google Technologies Integration

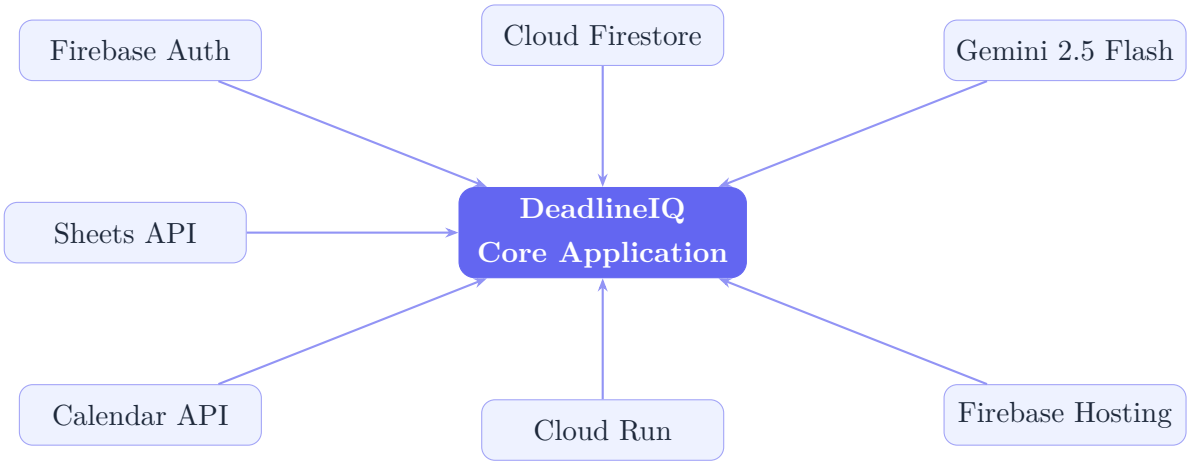
DeadlineIQ integrates **seven core Google technologies** spanning Cloud, Firebase, and Workspace products to power its intelligence and cloud infrastructure.

3.1 Integration Summary

Technology	Role in DeadlineIQ
Firebase Authentication	Google OAuth 2.0 identity management and session token validation for all protected operations.
Cloud Firestore	Real-time NoSQL database for user profiles, task documents, calendar slots, and behavioural metrics.
Gemini API (`gemini-2.5-flash`)	Drives NL task parsing, automatic subtask generation, and cognitive-load forecasting.
Google Calendar API	Syncs allocated focus slots and task deadlines to the user's primary Google Calendar.
Google Sheets API	Exports productivity logs and completed task data as Sheets-compatible CSV with OAuth sync validation.
Google Cloud Run	Pre-configured containerised deployments for serverless, auto-scaling production hosting.
Firebase Hosting	Global CDN delivery of production assets under verified SSL (`*.web.app / *.firebaseapp.com`).

Table 3.1: Google technology stack roles and responsibilities

3.2 Architecture Dependency Diagram



CHAPTER 4

Technical Architecture

4.1 Front-End Stack

Layer	Technology	Notes
UI Framework	React 18	Component-based SPA with concurrent rendering
Build Tool	Vite	Sub-second HMR; optimised production bundles
Styling	Tailwind CSS	Glassmorphism utilities + custom spatial transitions
Client ML	Pure JavaScript	MLP in `src/utils/localML.js`; no external lib
State / Sync	React Context + Firestore SDK	Real-time bi-directional sync
Routing	React Router v6	Client-side HTML5 history routing

Table 4.1: Front-end technology stack

4.2 Client-Side Neural Network (MLP)

The procrastination risk model is a **Multi-Layer Perceptron** written in pure JavaScript and executed entirely within the browser runtime. No training data or model weights ever leave the device.

4.2.1 Model Specification

Property	Value
Algorithm	Online supervised backpropagation
Training epochs	10 per update cycle
Output range	$p \in [0, 1]$ (procrastination probability)
Storage	`localStorage` (device-only)
Input Features	
Feature 1	Task urgency (normalised 0–1)
Feature 2	Estimated duration (hours)
Feature 3	Historical deferral rate
Feature 4	Time-of-day coefficient
Feature 5	Deadline proximity (hours remaining)

Table 4.2: MLP neural network specification

All neural network weights and training samples are stored exclusively in the user's browser `localStorage`. No behavioural telemetry is transmitted to any external endpoint.

4.3 Firestore Security Rules

All Firestore reads and writes are protected by production-grade security rules that enforce strict authenticated owner-only access:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/{document=**} {
      allow read, write:
        if request.auth != null
        && request.auth.uid == userId;
    }
  }
}
```

These rules guarantee that:

- Unauthenticated requests are rejected at the database layer.
- Each user can only access documents within their own `/users/{userId}/` namespace.
- Cross-user data leakage is architecturally impossible.

4.4 Project Directory Structure

```
deadlineiq/
 public/                    # Static assets (favicon, icons)
 src/
    components/
       Dashboard/          # HUD, telemetry, navigation
       Calendar/           # Glassmorphic weekly grid
       Kanban/             # Drag-and-drop progress board
       Forensics/          # Procrastination analytics
       Coach/              # IQ Coach co-pilot sidebar
       Habits/             # Streak tracker + gauges
       Settings/           # API keys and preferences
    utils/
       localML.js          # Client-side MLP neural network
    services/
       firebase.js         # Firestore and Auth SDK
       gemini.js           # Gemini API client wrapper
       calendar.js         # Google Calendar API wrapper
    App.jsx                 # Root component and routing
    main.jsx                # Vite entry point
 .env                       # Environment variables (NOT
    committed)
 firestore.rules            # Firestore security rules
 firebase.json              # Firebase Hosting configuration
 Dockerfile                 # Multi-stage container build
 nginx.conf                 # SPA routing fallback config
 vite.config.js             # Vite build configuration
 package.json               # Dependencies and scripts
```

CHAPTER 5

Local Development Setup

5.1 Prerequisites

Tool	Required Version	Notes
Node.js	v18 or v20	LTS releases strongly recommended
npm	Bundled with Node	Alternatively use `pnpm` or `yarn`
Git	Any recent release	For cloning the repository

Table 5.1: Development prerequisites

5.2 Installation Steps

5.2.1 Step 1 Navigate and Install Dependencies

```
# Navigate to the project directory
cd deadlineiq

# Install all npm dependencies
npm install
```

5.2.2 Step 2 Environment Configuration

Create a `.env` file in the project root. Populate it with your Firebase project credentials:

```
VITE_FIREBASE_API_KEY=your_firebase_api_key
VITE_FIREBASE_AUTH_DOMAIN=your_project_id.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your_project_id
VITE_FIREBASE_STORAGE_BUCKET=your_project_id.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=your_messaging_sender_id
VITE_FIREBASE_APP_ID=your_firebase_app_id
```

The Gemini Developer API Key can be entered directly in the web UI under **Settings** → **API Configuration**, so you do not need to add it to the `.env` file.

5.2.3 Step 3 Start the Development Server

```
npm run dev
```

Open `http://localhost:5174` in your browser. Vite's Hot Module Replacement (HMR) provides instant UI feedback during development.

CHAPTER 6

Deployment

6.1 Docker Deployment

DeadlineIQ ships with a multi-stage `Dockerfile` that compiles static assets and serves them via a custom `nginx.conf` with HTML5 history-mode fallback routing.

6.1.1 Build the Container Image

```
docker build -t deadlineiq .
```

6.1.2 Run the Container

```
# Maps container port 80 to host port 8080
docker run -d -p 8080:80 deadlineiq
```

Access the application at http://localhost:8080.

The container exposes port **80** internally (nginx default). Adjust `-p` *HOST_PORT*`:80` to fit your environment. Ensure no other service is bound to the chosen host port.

6.2 Firebase Hosting Deployment

```
# 1. Authenticate with your Google account
npx firebase login

# 2. Build optimised production assets
npm run build

# 3. Deploy to Firebase Hosting
npx firebase deploy --only hosting
```

After a successful deployment, the application is globally available at:

- `https://your-project-id.web.app`
- `https://your-project-id.firebaseapp.com`

Both endpoints are served over verified SSL with Firebase's global CDN.

6.3 Google Cloud Run Deployment

For serverless, auto-scaling production hosting, the project includes pre-configured Cloud Run artifacts. Integrate `cloudbuild.yaml` into your CI/CD pipeline to trigger automated container builds and deployments on every push to the main branch.

CHAPTER 7

Security Considerations

7.1 Authentication Layer

Firebase Authentication with Google OAuth 2.0 ensures every session is cryptographically signed and token-validated. Token expiry and refresh cycles are handled automatically by the Firebase SDK.

7.2 Data Isolation at the Database Layer

The Firestore namespace structure (`/users/{userId}/...`) combined with the security rules described in Section 4.3 makes cross-user data access architecturally impossible not merely a policy decision.

7.3 Client-Side ML Privacy

The MLP neural network runs entirely within the browser's JavaScript engine. Behavioural training data and learned weights are persisted only to `localStorage` on the user's device.

7.4 API Key Security Best Practices

- Add `.env` to `.gitignore` immediately. **Never commit credentials to version control.**
- In the Google Cloud Console, restrict Firebase API keys to authorised domains only (e.g., your `web.app` domain).
- Rotate Gemini API keys periodically from the Google AI Studio dashboard.
- Enable Firebase App Check to block unauthorised clients from accessing your backend.
- Review Firestore security rules before every production release.

Technical Documentation • v1.0 • June 2026

For configuration help, contribution guidelines, or bug reports, consult the repository `README.md` or contact the engineering team.